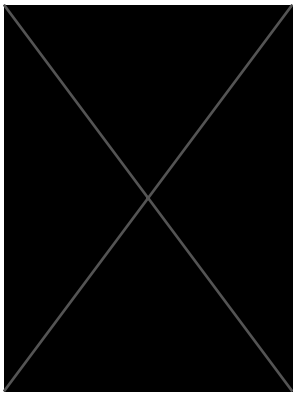
KT24403 OPERATING SYSTEM

SEMESTER II, 2023/2024

LECTURER: DR SALMAH BINTI FATTAH

LAB PROJECT

PREPARED BY:

BIL	NAME	MATRIC NUM
1	SYAZA SYAZANA BINTI ANWAR	
2	SHWEHTTA PARANI KUMAR	
3	LEENA A/P SELLAMANY	
4	WAN NURADRIANA BALQIS BT WAN AZMI	
5	NORSYLAH BINTI SAFRUDIN	

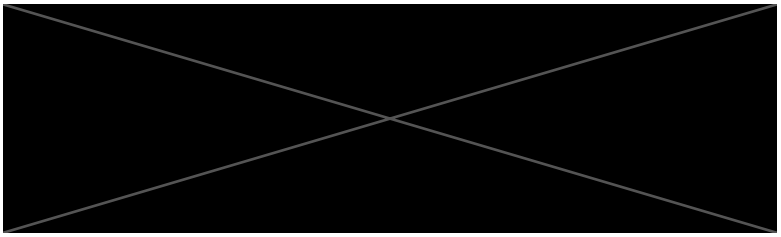


TABLE OF CONTENT

1.0 GROUP MEMBERS AND TASK DISTRIBUTION

BIL	NAME	TASK DISTRIBUTION
1	SYAZA SYAZANA BINTI ANWAR	Input/output handling and integration
2	SHWEHTTA PARANI KUMAR	Video presentation and testing
3	LEENA A/P SELLAMANY	Documentation and testing
4	WAN NURADRIANA BALQIS BT WAN AZMI	Algorithm design and implementation
5	NORSYLAH BINTI SAFRUDIN	Function implementation

2.0 CODE

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>

// Global variables for synchronization
std::mutex mutex_semaphore; // Mutex to protect shared resources for readers
std::mutex write_mutex;     // Mutex to ensure only one writer at a time
std::condition_variable cv; // Condition variable to notify waiting threads
int read_count = 0;         // Counter for the number of active readers
bool writer_active = false; // Flag to indicate if a writer is active

// Function to wait until the condition is met
void wait(std::unique_lock<std::mutex>& lock) {
    cv.wait(lock);
}

// Function to signal waiting threads
void signal() {
    cv.notify_all();
}

// Function for readers
void Read() {
    // Acquire the mutex for readers
    std::unique_lock<std::mutex> lock(mutex_semaphore);

    // Wait if a writer is active
    while (writer_active) {
        std::cout << "***You are a Reader.***\n";
```

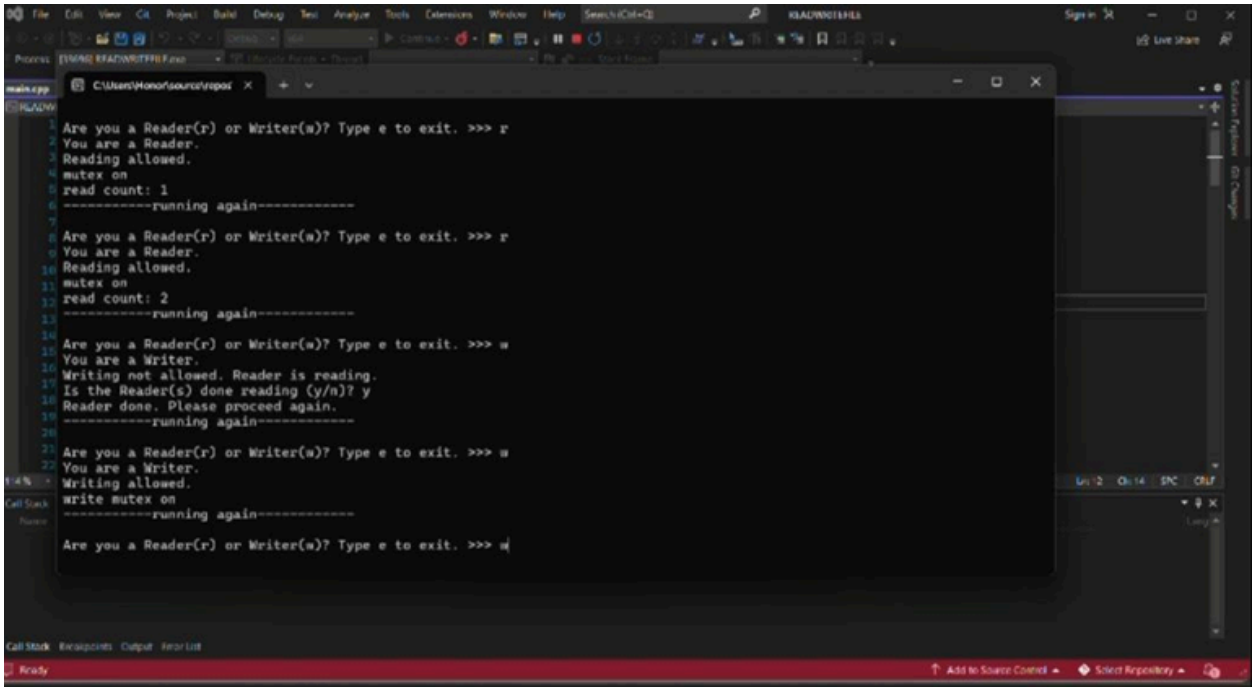
```
        std::cout << "Reading not allowed. Writer is writing.\n";
        std::cout << "Is the Writer done writing (y/n)? ";
        char input;
        std::cin >> input;
        if (input == 'y') {
            // Notify all waiting threads if the writer is done
            writer_active = false;
            signal();
            std::cout << "Writer done. Please proceed again.\n";
            return;
        } else {
            std::cout << "Writer not done. Please proceed again.\n";
            return;
        }
    }

    // Increment the reader count and allow reading
    ++read_count;
    std::cout << "***You are a Reader.***\n";
    std::cout << "Reading allowed.\n";
    std::cout << "mutex on\n";
    std::cout << "read count: " << read_count << "\n";
}

// Function for writers
void Write() {
    // Acquire the mutex to ensure only one writer
    std::unique_lock<std::mutex> write_lock(write_mutex);

    // Acquire the mutex to check for active readers or writers
    std::unique_lock<std::mutex> read_lock(mutex_semaphore);
    while (writer_active || read_count > 0) {
```

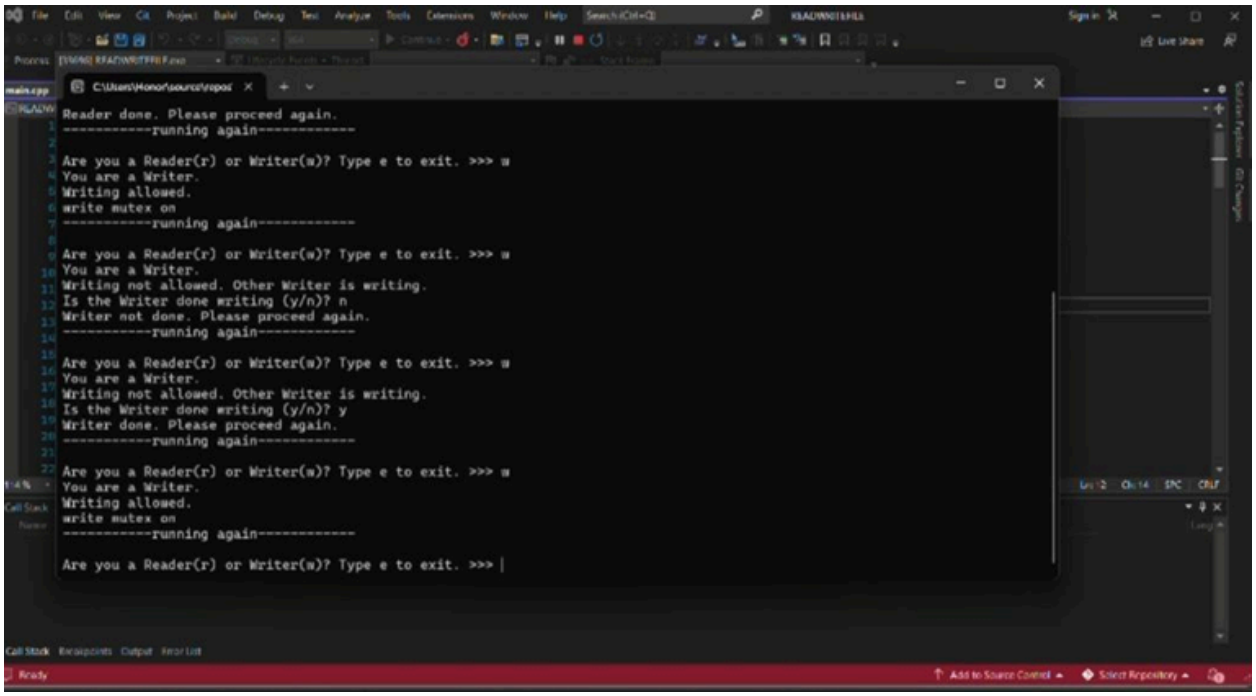
```
if (writer_active) {
    std::cout << "***You are a Writer.***\n";
    std::cout << "Writing not allowed. Other Writer is writing.\n";
    std::cout << "Is the Writer done writing (y/n)? ";
    char input;
    std::cin >> input;
    if (input == 'y') {
        // Notify all waiting threads if the writer is done
        writer_active = false;
        signal();
        std::cout << "Writer done. Please proceed again.\n";
        return;
    } else {
        std::cout << "Writer not done. Please proceed again.\n";
        return;
    }
} else if (read_count > 0) {
    std::cout << "***You are a Writer.***\n";
    std::cout << "Writing not allowed. Reader is reading.\n";
    std::cout << "Is the Reader(s) done reading (y/n)? ";
    char input;
    std::cin >> input;
    if (input == 'y') {
        // Notify all waiting threads if the readers are done
        read_count = 0;
        signal();
        std::cout << "Reader done. Please proceed again.\n";
        return;
    } else {
        std::cout << "Reader not done. Please proceed again.\n";
        return;
    }
```

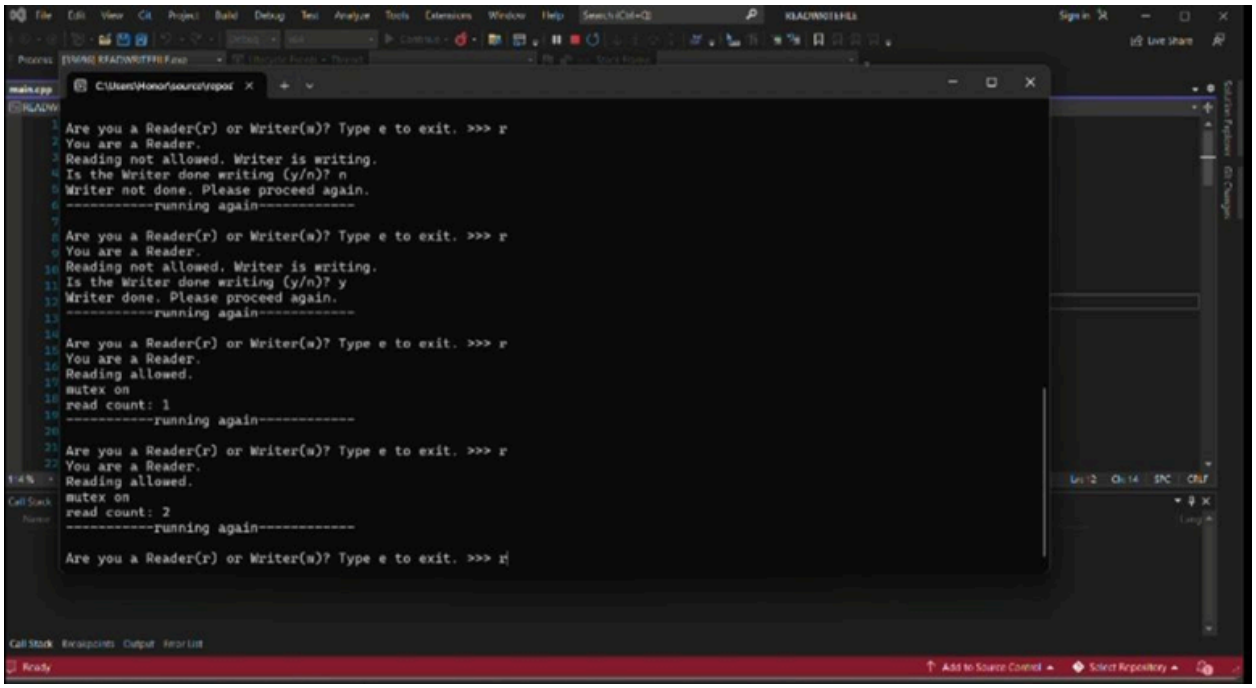
```
        }
    }

    // Allow writing and set writer_active flag
    writer_active = true;
    std::cout << "***You are a Writer.***\n";
    std::cout << "Writing allowed.\n";
    std::cout << "write mutex on\n";
}

// Main function to simulate reader and writer interactions
int main() {
    std::cout << "You are currently accessing A file.";
    char choice;
    while (true) {
        // Prompt the user for reader or writer role
        std::cout << "\nAre you a Reader(r) or Writer(w)? Type e to exit. >>> ";
        std::cin >> choice;

        if (choice == 'e') {
            // Exit the program
            std::cout << "Process exited.\n";
            break;
        }

        // Call the appropriate function based on user input
        if (choice == 'r') {
            Read();
        } else if (choice == 'w') {
            Write();
        }
```

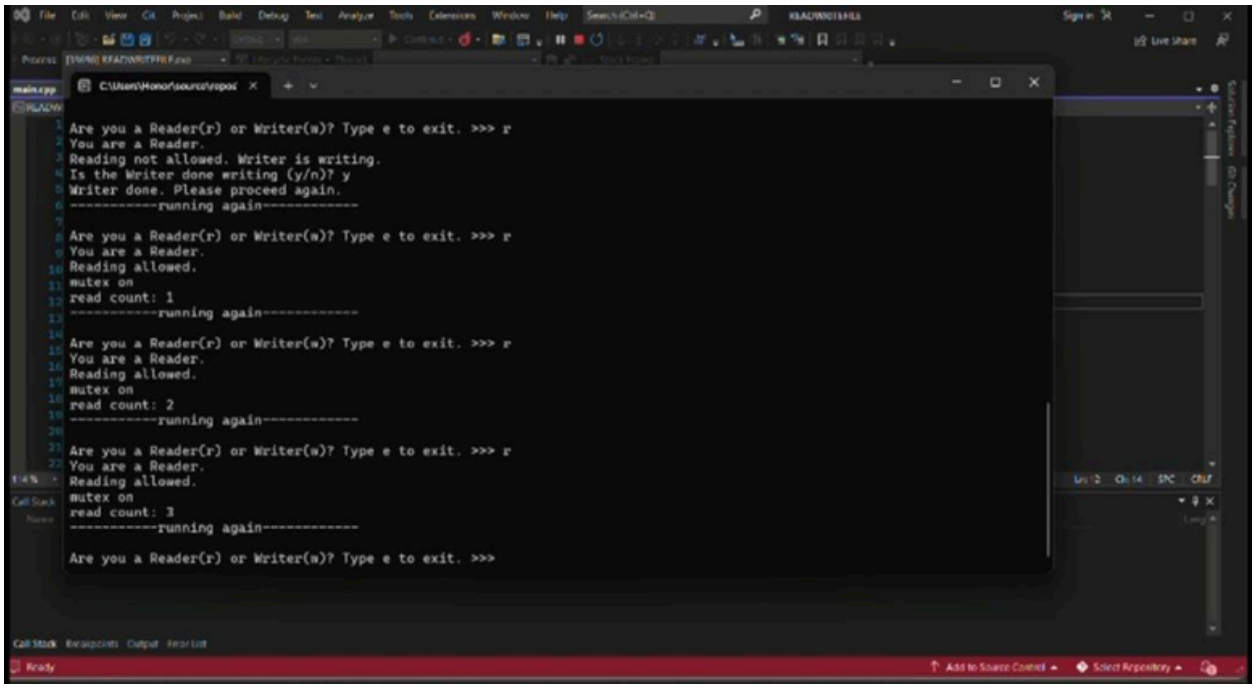
```
        std::cout << "-----------running again------------\n";
    }

    return 0;
}
```

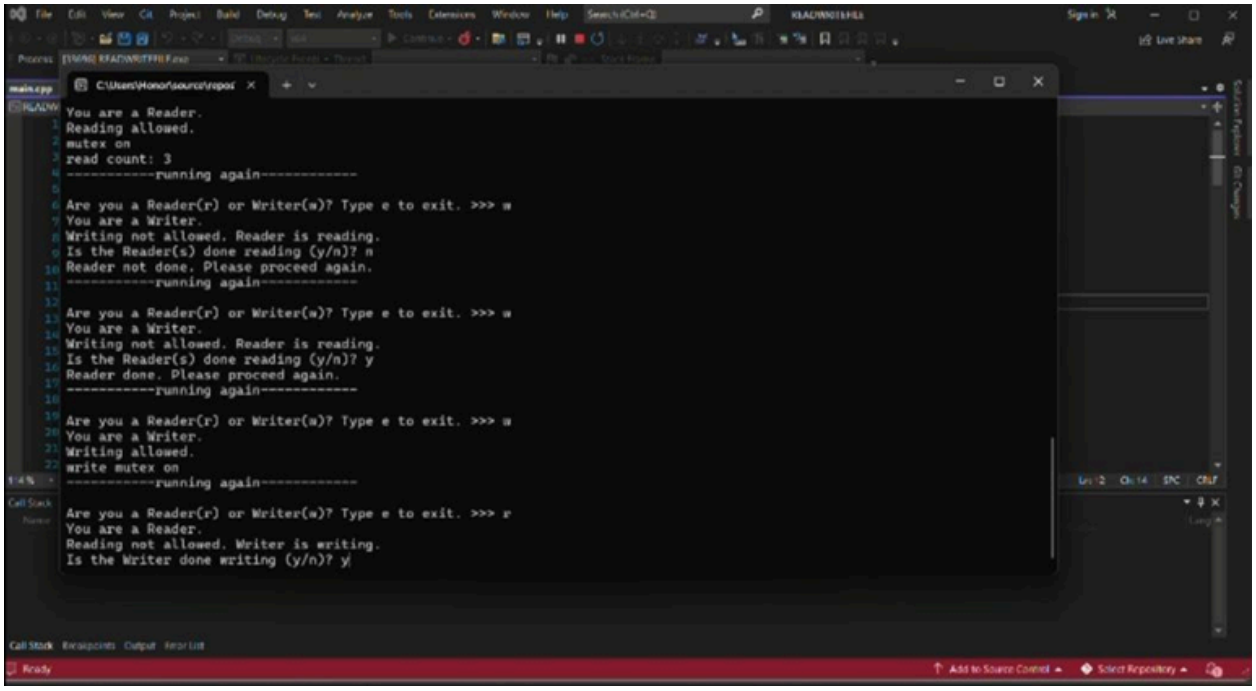
3.0 SCREENSHOTS OF TEST SCENARIOS

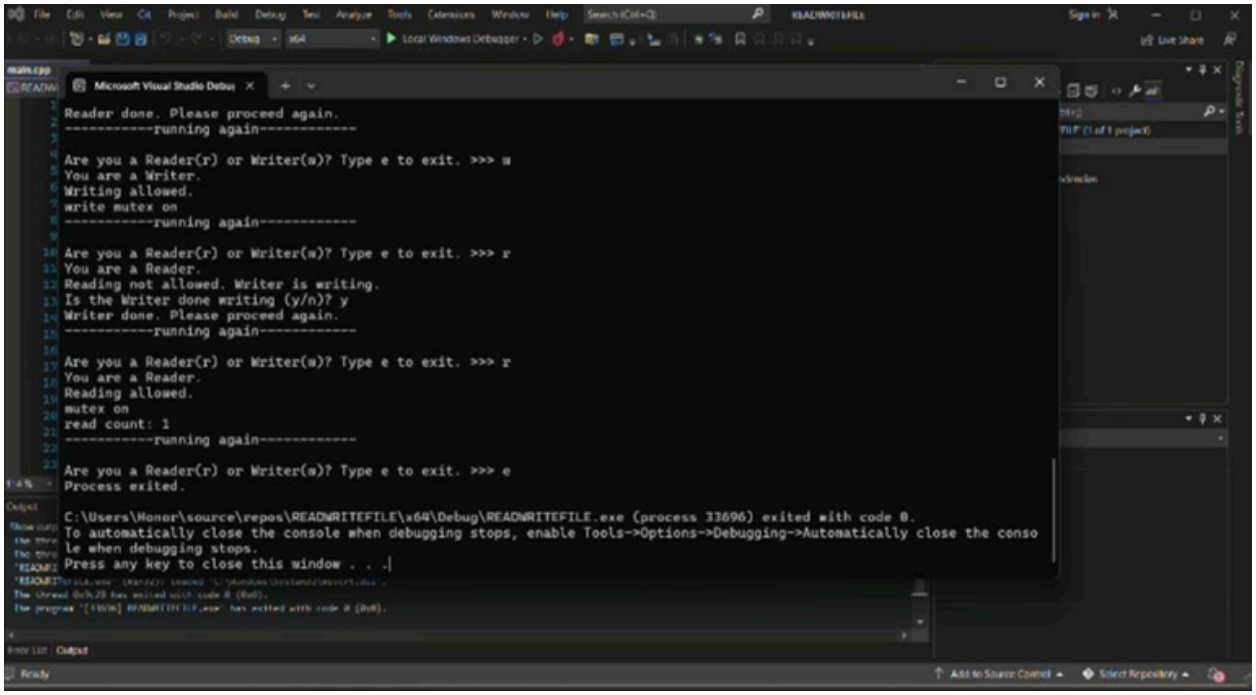





Are you a Reader(r) or Writer(w)? Type e to exit. >>> r
You are a Reader.
Reading not allowed. Writer is writing.
Is the Writer done writing (y/n)? n
Writer not done. Please proceed again.
----------running again----------
Are you a Reader(r) or Writer(w)? Type e to exit. >>> r
You are a Reader.
Reading not allowed. Writer is writing.
Is the Writer done writing (y/n)? y
Writer done. Please proceed again.
----------running again----------
Are you a Reader(r) or Writer(w)? Type e to exit. >>> r
You are a Reader.
Reading allowed.
mutex on
read count: 1
----------running again----------
Are you a Reader(r) or Writer(w)? Type e to exit. >>> r
You are a Reader.
Reading allowed.
mutex on
read count: 2
----------running again----------
Are you a Reader(r) or Writer(w)? Type e to exit. >>> r


Are you a Reader(r) or Writer(w)? Type e to exit. >>> r
You are a Reader.
Reading not allowed. Writer is writing.
Is the Writer done writing (y/n)? y
Writer done. Please proceed again.
----------running again----------
Are you a Reader(r) or Writer(w)? Type e to exit. >>> r
You are a Reader.
Reading allowed.
mutex on
read count: 1
----------running again----------
Are you a Reader(r) or Writer(w)? Type e to exit. >>> r
You are a Reader.
Reading allowed.
mutex on
read count: 2
----------running again----------
Are you a Reader(r) or Writer(w)? Type e to exit. >>> r
You are a Reader.
Reading allowed.
mutex on
read count: 3
----------running again----------
Are you a Reader(r) or Writer(w)? Type e to exit. >>>


You are a Reader.
Reading allowed.
mutex on
read count: 3
----------running again----------
Are you a Reader(r) or Writer(w)? Type e to exit. >>> w
You are a Writer.
Writing not allowed. Reader is reading.
Is the Reader(s) done reading (y/n)? n
Reader not done. Please proceed again.
----------running again----------
Are you a Reader(r) or Writer(w)? Type e to exit. >>> w
You are a Writer.
Writing not allowed. Reader is reading.
Is the Reader(s) done reading (y/n)? y
Reader done. Please proceed again.
----------running again----------
Are you a Reader(r) or Writer(w)? Type e to exit. >>> w
You are a Writer.
Writing allowed.
write mutex on
----------running again----------
Are you a Reader(r) or Writer(w)? Type e to exit. >>> r
You are a Reader.
Reading not allowed. Writer is writing.
Is the Writer done writing (y/n)? y


Reader done. Please proceed again.
----------running again----------
Are you a Reader(r) or Writer(w)? Type e to exit. >>> w
You are a Writer.
Writing allowed.
write mutex on
----------running again----------
Are you a Reader(r) or Writer(w)? Type e to exit. >>> r
You are a Reader.
Reading not allowed. Writer is writing.
Is the Writer done writing (y/n)? y
Writer done. Please proceed again.
----------running again----------
Are you a Reader(r) or Writer(w)? Type e to exit. >>> r
You are a Reader.
Reading allowed.
mutex on
read count: 1
----------running again----------
Are you a Reader(r) or Writer(w)? Type e to exit. >>> e
Process exited.
C:\Users\Honor\source\repos\READWRITEFILE\x64\Debug\READWRITEFILE.exe (process 33696) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .